Lund University

Course code: FMAN20
Course name: Image Analysis

Assignment 4

Elina Rosato

Program: MSc Program in Machine Learning, Systems and Control
Lecturer: K. Magnus J. Oskarsson

October 25, 2026

1 Color correction of images

1.1 Method

The image $\boldsymbol{g}$ has a color cast and the image $\boldsymbol{f}$ has the correct colors. The color change is a linear transformation, so for each pixel $\boldsymbol{g}(i,j) = A\,\boldsymbol{f}(i,j)$, where $\boldsymbol{f}(i,j)$ and $\boldsymbol{g}(i,j)$ are RGB vectors with 3 numbers and A is an unknown 3×3 matrix. When A is found we get an estimate of the original image from $\hat{\boldsymbol{f}} = A^{-1}\boldsymbol{g}$. Since some values can end up outside $[0,1]$, I clip them to that range.

The matrix A has 9 unknowns. One pixel gives 3 equations (red, green and blue), so we need $9/3 = 3$ pixels. **The minimum is 3 corresponding pixels.** If the three colors of $\boldsymbol{f}$ are the columns of F and the three colors of $\boldsymbol{g}$ are the columns of G, then $G = AF$ and $A = GF^{-1}$. This needs F to be invertible, which is not true if the three colors are almost the same.

The equation $\boldsymbol{g} = A\boldsymbol{f}$ is not true for all pixels. The black squares are in `task1_org.png` (that is $\boldsymbol{f}$), not in `task1_color.png` as the task text says. Inside a square $\boldsymbol{f} = (0,0,0)$ but $\boldsymbol{g}$ has a normal color. These pixels are 6.7% of the image and the other 93.3% I call the uncorrupted pixels. If a bad pixel is picked we get a bad A, so I used RANSAC, which tries many random samples and keeps the best one. I repeat 200 times:

1. Pick 3 random pixels and calculate $A = GF^{-1}$ (samples where F is almost not invertible are skipped).
2. For every pixel, calculate the error $\|\boldsymbol{g} - A\boldsymbol{f}\|$. If it is smaller than a threshold τ, the pixel is an inlier. Count the inliers.

The A with the most inliers is kept. Finally I calculate A again with least squares on all of its inliers, which is more exact than using only 3 pixels since the pixel values are rounded and a bit noisy.

1.2 Choice of threshold

I tried four values of τ (pixel values are between 0 and 1):

τ	Inliers	PSNR on uncorrupted pixels
0.02	14.2%	32.94
0.05	83.2%	33.34
0.10	93.7%	33.02
0.20	94.0%	33.02

With $\tau = 0.02$ and $\tau = 0.05$ the inliers are fewer than the 93.3% uncorrupted pixels, so good pixels are thrown away because of the noise. With $\tau = 0.1$ the inliers are 93.7%, close to the real fraction, and $\tau = 0.2$ gives almost the same result. I chose $\tau = 0.1$ for this reason and not for the PSNR.

1.3 Results

The estimated matrix is

$$A = \begin{pmatrix} 0.974 & -0.107 & 0.171 \\ 0.117 & 0.774 & -0.164 \\ -0.132 & 0.208 & 0.865 \end{pmatrix}, \tag{1}$$

and 2 857 020 of 3 048 192 pixels (93.7%) were inliers. The code is in `task1.py`.

The purple color is gone and $\hat{\boldsymbol{f}}$ looks like $\boldsymbol{f}$. In the black squares $\hat{\boldsymbol{f}}$ shows the real image, since it is calculated from $\boldsymbol{g}$.



Figure 1: Left: $\boldsymbol{g}$ with the color cast. Middle: my estimate $\hat{\boldsymbol{f}} = A^{-1}\boldsymbol{g}$ (saved as `task1_corrected.png`). Right: the original $\boldsymbol{f}$ with the black squares.

I compared $\hat{\boldsymbol{f}}$ with $\boldsymbol{f}$ using PSNR and SSIM (from `scikit-image`) and ꟻLIP (from `flip-evaluator`), and I did the same for $\boldsymbol{g}$ to see the improvement. PSNR is based on the mean squared difference of the pixels, SSIM compares small windows (brightness, contrast, structure) and ꟻLIP measures how big the difference looks to a human. For SSIM and ꟻLIP I took the mean of the error map, either over the whole image or only over the uncorrupted pixels (a mask that is true where $\boldsymbol{f}$ is not black).

	Metric	Whole image	Uncorrupted pixels
$\hat{\boldsymbol{f}}$ vs. $\boldsymbol{f}$	PSNR (dB, higher is better)	16.37	33.02
	SSIM (higher is better)	0.736	0.788
	ꟻLIP (lower is better)	0.118	0.070
$\boldsymbol{g}$ vs. $\boldsymbol{f}$	PSNR (dB)	15.43	19.70
	SSIM	0.740	0.792
	ꟻLIP	0.537	0.464

1.4 Discussion

On the whole image the PSNR only goes from 15.4 to 16.4 dB, but on the uncorrupted pixels it goes from 19.7 to 33.0 dB. The reason is the black squares: there $\boldsymbol{f}$ is black and $\hat{\boldsymbol{f}}$ has the real colors, so the error is large even though A is not wrong. These pixels are only 6.7% of the image but they dominate the mean, so the numbers for the uncorrupted pixels show how good the color correction is. ꟻLIP agrees, the error goes from 0.464 to 0.070.

SSIM did not improve (0.792 for $\boldsymbol{g}$, 0.788 for $\hat{\boldsymbol{f}}$). A possible reason is that SSIM looks at structure like edges and textures and a color cast does not change this much, but I did not test it.

2 Segmentation with graph cuts

2.1 Mean and standard deviation

I assume that the intensities in each class follow a Gaussian distribution, and I estimated the mean μ and standard deviation σ from the known values (2362 chamber values and 6854 background values):

	Mean μ	Standard deviation σ
Chamber	0.3542	0.0992
Background	0.1065	0.0936

2.2 Negative log likelihoods

For a Gaussian, $P(f) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(f-\mu)^2/(2\sigma^2)}$, so

$$-\log P(f) = \log\bigl(\sqrt{2\pi}\,\sigma\bigr) + \frac{(f-\mu)^2}{2\sigma^2}. \tag{2}$$

The first term comes from the normalizing constant and I leave it out, as the task says (it is almost the same for the two classes since the σ are almost equal). What is left is

$$-\log P(f \,|\, \text{chamber}) = \frac{(f-\mu_{\text{chamber}})^2}{2\sigma^2_{\text{chamber}}}, \qquad -\log P(f \,|\, \text{background}) = \frac{(f-\mu_{\text{background}})^2}{2\sigma^2_{\text{background}}}. \tag{3}$$

These are small if the intensity is close to the mean of the class, so they work as the cost of giving a pixel each label.

2.3 The graph

Every pixel is a node, and there is a source s (the chamber side) and a sink t (the background side). The edges are:

- between each pixel and its 4 neighbours, with weight ν. If two neighbours get different labels an edge is cut, so these edges give ν times the length of the boundary;
- from the source to each pixel, with weight $-\log P(f \,|\, \text{background})$. It is cut if the pixel is on the sink side, i.e. if the pixel is called background;
- from each pixel to the sink, with weight $-\log P(f \,|\, \text{chamber})$. It is cut if the pixel is called chamber.

Every cut is a labeling of the image, and its cost is the energy (4) in the task, so the minimum cut gives the best labeling. The minimum cut is found with the maximum flow from the source to the sink (`scipy`). The pixels that can still be reached from the source after the max flow are the chamber pixels. The weights were multiplied by 10000 and rounded, since the max flow algorithm needs integers. The code is in `task2.py`.

2.4 Choice of ν and result

Figure 2 shows the result for several ν. With $\nu = 0$ every pixel just takes the cheaper label, which gives many small spots. When ν grows the spots disappear and the boundary becomes smoother, but with a large ν it becomes too stiff and cuts away parts of the chambers (see $\nu = 8$). I chose $\nu = 3$ as a compromise.

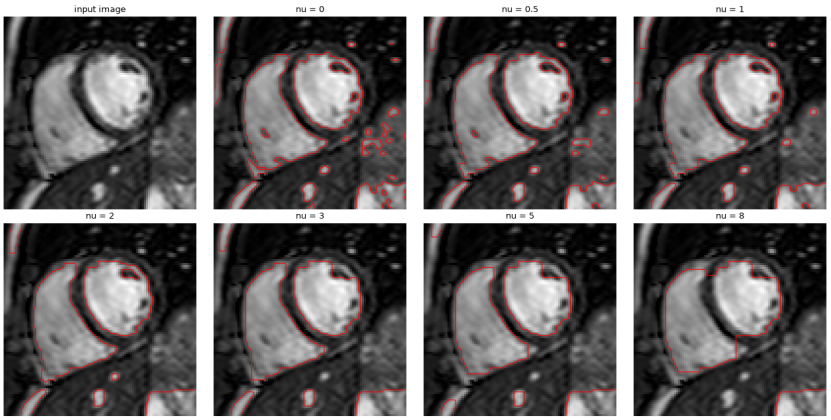


Figure 2: The segmentation (red line) for different values of ν.

Figure 3 shows the input and the result for $\nu = 3$.

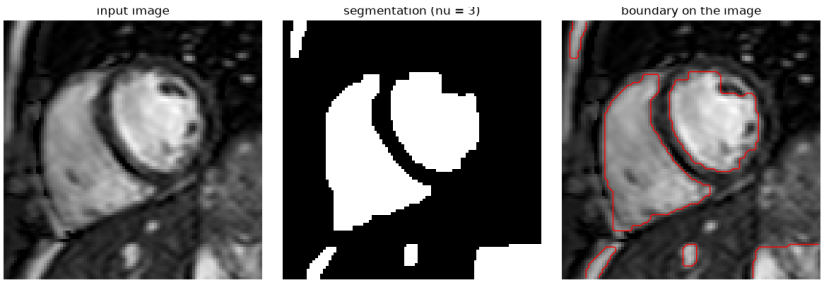


Figure 3: Left: the input image. Middle: the segmentation with $\nu = 3$ (white is chamber). Right: the boundary on the image.

Both chambers are found and the boundary follows them well. The result is not perfect: other bright parts, for example the bottom right corner and a strip at the left edge, are also labeled as chamber. Their intensities are like the chambers and the method only uses intensity and boundary length, so it cannot tell the difference. I did not do the optional part 3.

3 Computer vision

A 3D point $U = (X, Y, Z)$ is projected to the image by writing it with an extra 1, $(X, Y, Z, 1)^T$, and multiplying with the camera matrix P_1. This gives three numbers $(a, b, c)^T = P_1(X, Y, Z, 1)^T$, and the image point is $(a/c,\ b/c)$. A point can be the 3D point of the detected point $u = (6, -\frac{1}{2})$ only if this gives exactly u. I calculate this for the three points, with

$$P_1 = \begin{pmatrix} 1 & 2 & 3 & 3 \\ 3 & 2 & 0 & -2 \\ 1 & 1 & 2 & 0 \end{pmatrix}. \tag{4}$$

$U_1 = (2, \frac{1}{2}, 1)$:

$$P_1 \begin{pmatrix} 2 \\ \frac{1}{2} \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2+1+3+3 \\ 6+1+0-2 \\ 2+\frac{1}{2}+2+0 \end{pmatrix} = \begin{pmatrix} 9 \\ 5 \\ \frac{9}{2} \end{pmatrix} \Rightarrow \left(\frac{9}{9/2}, \frac{5}{9/2}\right) = \left(2, \tfrac{10}{9}\right) \neq u. \tag{5}$$

$U_2 = (\frac{1}{3}, \frac{1}{3}, 0)$:

$$P_1 \begin{pmatrix} \frac{1}{3} \\ \frac{1}{3} \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{3}+\frac{2}{3}+0+3 \\ 1+\frac{2}{3}+0-2 \\ \frac{1}{3}+\frac{1}{3}+0+0 \end{pmatrix} = \begin{pmatrix} 4 \\ -\frac{1}{3} \\ \frac{2}{3} \end{pmatrix} \Rightarrow \left(\frac{4}{2/3}, \frac{-1/3}{2/3}\right) = \left(6, -\tfrac{1}{2}\right) = u. \tag{6}$$

$U_3 = (-12, 18, 1)$:

$$P_1 \begin{pmatrix} -12 \\ 18 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -12+36+3+3 \\ -36+36+0-2 \\ -12+18+2+0 \end{pmatrix} = \begin{pmatrix} 30 \\ -2 \\ 8 \end{pmatrix} \Rightarrow \left(\frac{30}{8}, \frac{-2}{8}\right) = \left(\tfrac{15}{4}, -\tfrac{1}{4}\right) \neq u. \tag{7}$$

Answer: only $U_2 = (\frac{1}{3}, \frac{1}{3}, 0)$ can project to the detected point u. It is the only one of the three points that gives exactly $(6, -\frac{1}{2})$ after the division by the third coordinate.

4 The garment classification system, part 4

4.1 Version 1

Version 1 is the system from Assignment 3. `im2segment` thresholds the image at 20 and finds the connected components, `segment2feature` gives 10 shape numbers (for example the aspect ratio and the number of holes), and `feature2class` is a k-nearest-neighbour classifier trained on the 1000 training segments (it chose $k = 1$). I ran the benchmark script on the five datasets. The hitrates (the fraction of the garments that were classified correctly) were:

Dataset	Version 1	Version 2
short1	60.0%	82.0%
short2	30.0%	74.0%
home1	48.9%	79.4%
home2	50.1%	77.6%
home3	44.3%	77.8%

Version 1 did not go well. Only short1 (60.0%) and home2 (50.1%) are above 50%, and short2 is only 30.0%. To find out why, I measured the two main steps separately. The benchmark pairs the i-th segment with the i-th ground-truth garment, so the segmentation must find exactly the right garments in the right order. The table shows the mean Jaccard index of the segmentation, in how many images the number of segments was right ("Right no."), and the hitrate if the classifier gets the ground-truth segments ("True segm."):

	Version 1			Version 2		
Dataset	Jaccard	Right no.	True segm.	Jaccard	Right no.	True segm.
short1	0.945	100%	80.0%	0.946	100%	78.0%
short2	0.775	70%	68.0%	0.941	100%	74.0%
home1	0.896	87%	68.7%	0.955	100%	78.8%
home2	0.894	88%	71.9%	0.955	100%	80.6%
home3	0.862	82.5%	66.1%	0.956	100%	78.1%

For version 1 there are two problems. First, the segmentation: a fixed threshold of 20 does not work for dark garments or dim parts of a garment, which are broken into pieces or lose parts (short2 has many dark garments, and there the right number of segments was found in only 70% of the images). One extra or missing segment moves all later garments one position, so all of them become wrong, which is why short2 is so bad (Figure 4 shows an example). This costs 20 to 38 percentage points (compare the hitrate with the hitrate on the true segments). Second, the classifier: even with perfect segments it only reaches 66–80%. The 10 hand-made features are on very different scales, which is not good for a distance based method, and the classes are similar (a pullover, a coat and a shirt have almost the same silhouette).

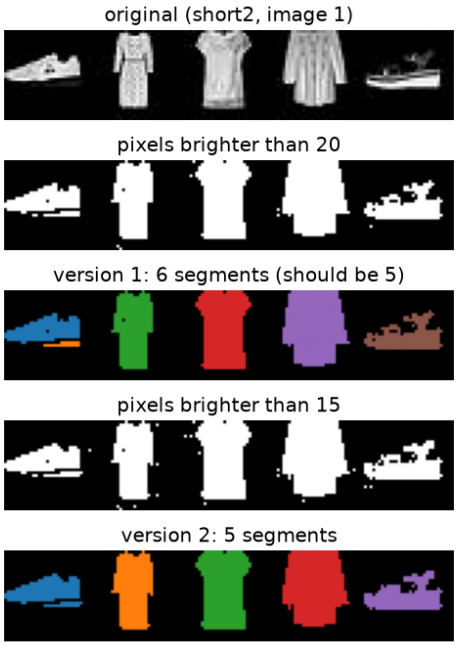


Figure 4: An image from short2. Rows from top to bottom: the original; the pixels brighter than 20; the segments of version 1 (one color per segment); the pixels brighter than 15; the segments of version 2. The sole of the shoe is darker than 20, so version 1 makes it a separate segment and finds 6 segments instead of 5. Version 2 keeps the shoe in one piece.

4.2 Version 2

I changed all three steps.

- **Segmentation.** I lowered the threshold to 15, removed tiny specks and filled holes in the garments, so that dark garments stay in one piece. Then I use that every image has 5 garments in a row: I look at which columns of the image contain garment pixels, and cut the image at the 4 widest empty gaps. This always gives exactly 5 segments in the right (left to right) order. If there are too few gaps (garments that touch) the code uses connected components instead, but this was never needed on the five datasets. The threshold is not sensitive: values from 10 to 20 gave similar Jaccard indices.

- **Features.** Instead of the 10 numbers I use the silhouette of the garment: cut out the bounding box, put it in the middle of a square (so the shape is not stretched) and scale it to 20×20 pixels, plus the aspect ratio. This gives 401 numbers and does not depend on the size of the garment, which matters since the garments in the home datasets are up to 70 pixels high but the training garments are 28.

- **Classifier.** I use a support vector machine with an RBF kernel, after standardizing every feature. I tested other classifiers (k-nearest-neighbour, logistic regression, random forest, LDA) with 5-fold cross-validation on the 1000 training segments, and the SVM was among the best (80.6% against 74.2% for version 1 with the old features). I set $C = 10$ in the same way. I did not tune the classifier on the five datasets.

The results are in the table above. Version 2 reaches 74 to 82% on all five datasets, so the aim of more than 50% is reached everywhere. The segmentation now finds 5 segments in every image and the Jaccard index is about 0.95 on all datasets. The difference between the hitrate on the true segments and on my own segments is now at most about 3 percentage points (on

short1 it is even 4 points higher, which is just chance with 50 garments), so the segmentation does not cost much any more. The classifier also became better, from 66–80% to 74–81% with perfect segments, and this is now what limits the system.

4.3 Remaining errors and limitations

Over all 3100 garments in the five datasets, version 2 got 78.3% right (version 1 got 47.7%). Most of the errors are between garments that have similar silhouettes: pullover and coat are mixed up, and shirts are often classified as coat, pullover or T-shirt (only 39% of the shirts are correct, against 92% for trousers and 95% for ankle boots). The system only sees the shape of the segment, not the texture or the colors, so these classes are hard to separate. A way to improve this would be to give the classifier information from the gray values of the garment, but the benchmark only gives the segment to `segment2feature`, so I did not do it.

The segmentation uses that there are always exactly 5 garments in a row, which is true for these datasets but would not hold for other layouts. The datasets with only 10 images (short1 and short2) have 50 garments each, so their hitrates are uncertain by about ±6 percentage points. I chose the threshold 15 after looking at the Jaccard index on these datasets, but since the result is almost the same from 10 to 20 I do not think this matters.